AI224: Operating Systems — Lecture 7

Dr. Karim Lounis

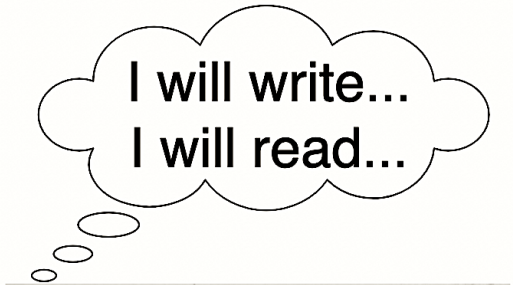
Jan - May 2025

Classical Synchronization Problems

Classical Synchronization Problems

Readers & Writers Problem

Writers



Waiting Queue

Readers

Readers & Writers Problem

This problem consists of: (1) A set of reader threads $R = \{r_1, \ldots, r_n\}$ & (2) a set of writer threads $W = \{w_1, \ldots, w_m\}$. Both readers and writers share a data structure D (e.g., file, database, variable, ...) such that:

- Multiple readers $R' \subseteq R$ can read at the same time.
- Only one writer $w_i \in W$ can write at a time.
- If $\exists r \in R$ that is reading, new writers have to wait (no preemption).
- If $\exists w \in W$ that is writing, new writers and readers have to wait.
- If $\exists r \in R$ that is reading, new readers can start reading.

There exist three solutions: Readers have priority, Writers have priority, and starvation free solution.

Let's discuss the algorithm together...

Readers & Writers Problem ($| R |=| W |= 1$)

One reader $R = \{r\}$ and one writer $W = \{w\}$.

```
Semaphore m = 1;
```

r()

```
{
   while(1)
   {
      acquire(m);
      read();
      release(m);
      do_something_else();
   }
}
```

w()

```
{
    while(1)
    {
       acquire(m);
       write();
       release(m);
       do_something_else();
    }
}
```

Readers & Writers Problem ($| R | \geq | W | = m \geq 1$)

Multiple readers $R = \{r_1, \ldots, r_n\}$ and multiple writers $W = \{w_1, \ldots, w_m\}$.

1. Readers have priority. Let us discuss the solution where the Readers have priority.

Readers have priority means: when a new reader arrives and a writer is already waiting for previous readers to terminate, then the new reader **can** start reading with the other readers without waiting.

This solution is not a starvation-free solution.

e.g., If there is a steady stream of readers, writers may not get the chance to write.

Let's discuss the algorithm together...

Readers & Writers problem ($| R | \geq | W | = m \geq 1$)

Multiple readers $R = \{r_1, \ldots, r_n\}$ and multiple writers $W = \{w_1, \ldots, w_m\}$.

2. Writers have priority. This will be in **Lab 8**.

Writers have priority means: when a new reader arrives and a writer is already waiting for previous readers to terminate, then the new reader **cannot** start reading along with the other readers but has to wait.

AND

When there is waiting readers and writers during a writing, writers have the priority to be waken up over the readers at the end of the writing.

This solution is not a starvation-free solution.

i.e., If there is a steady stream of writers, readers may not get the chance to read.

Let's discuss the algorithm together...

Readers & Writers problem ($| R | \geq | W | = m \geq 1$)

Multiple readers $R = \{r_1, \ldots, r_n\}$ and multiple writers $W = \{w_1, \ldots, w_m\}$.

3. Starvation-free. Let us briefly discuss this solution.

i.e., Readers yield for writers and writers yield for readers

Basically

When a reader shows up, it cannot start if a writer is waiting

and

When writers are waiting, one writer can start after the last current reader finishes

This solution is starvation-free.

i.e., Busy system: ..., one writer, batch of readers, one writer, ...

Dinning Philosophers Problem

Dinning Philosophers Problem

Dinning Philosophers Problem (by Dijkstra 1965)

It was originally formulated in 1965 by Edsger Dijkstra as a student exam exercise, presented in terms of computers competing for access to tape drive peripherals.

Dinning Philosophers Problem

A set of five philosophers sitting around a table $P = \{p_0, p_1, p_2, p_3, p_4\}$ and five forks $F = \{f_0, f_1, f_2, f_3, f_4\}$. Each philosopher P_i has one fork on his left side and one fork on his right side s.t:

- Each philosopher has a plate of spaghetti in front of him.
- A philosopher spends his time alternating between thinking & eating.
- Another version: Philosophers eating rice using chopsticks.
- A philosopher P_i needs both forks (f_i & f_{i+1}) to be able to eat.
- If a philosopher grabs one fork, it does not put it down till getting the second fork and eating (**Hold and wait property**).
- A philosopher is not allowed to take a fork from another philosopher's hands (**No preemption property**).
- A philosopher cannot take both forks at the same time.
- Parallelism increases with the number of philosophers.

Dinning Philosophers Problem

There exist multiple solutions, we discuss some of them.

Solution 1: Pick up a fork on the right side and then pick up another fork on the left side, then start eating the food.

Philosopher(Integer i)

```
{
   while(1)
   {
      Think();
         ...
      Eat();
          ...
   }
}
```

Dinning Philosophers Problem

Solution 1: Pick up a fork on the right side and then pick up another fork on the left side, then start eating the food.

`Semaphore Fork [5]; /*` $\forall i \in \{0, 1, 2, 3, 4\}$ `Fork[i]=1 */`

Philosopher(Integer i)

```
{
   while(1)
   {
      Think();
      acquire(Fork[i]);
      acquire(Fork[(i+1) mod 5]);
      Eat();
      release(Fork[i]);
      release(Fork[(i+1) mod 5]);
   }
}
```

No two adjacent philosophers will eat at the same time.

Dinning Philosophers Problem

Solution 1: Pick up a fork on the right side and then pick up another fork on the left side, then start eating the food.

`Semaphore Fork [5]; /*` $\forall i \in \{0,1,2,3,4\}$ `Fork[i]=1 */`

Philosopher(Integer i)

```
{
   while(1)
   {
      Think();
      acquire(Fork[i]);
      acquire(Fork[(i+1) mod 5]);
      Eat();
      release(Fork[i]);
      release(Fork[(i+1) mod 5]);
   }
}
```

However, this solution is not a deadlock-free solution $\Longrightarrow$ starvation.

Dinning Philosophers Problem

Solution 2: A mix of left handed and right handed philosophers.

`Semaphore Fork [5]; /*` $\forall i \in \{0,1,2,3,4\}$ `Fork[i]=1 */`

Philosopher(integer i)

```
{
   while(1)
   {
      Think();
      acquire(Fork[Min(i, (i+1) mod 5)]);
      acquire(Fork[Max(i, (i+1) mod 5)]);
      Eat();
      release(Fork[Min(i, (i+1) mod 5)]);
      release(Fork[Max(i, (i+1) mod 5)]);
   }
}
```

Actually, we are setting up a global ordering as per the request of resources.

Dinning Philosophers Problem

Solution 2: A mix of left handed and right handed philosophers.

`Semaphore Fork [5]; /*` $\forall i \in \{0,1,2,3,4\}$ `Fork[i]=1 */`

Philosopher(integer i)

```
{
   while(1)
   {
      Think();
      acquire(Fork[Min(i, (i+1) mod 5)]);
      acquire(Fork[Max(i, (i+1) mod 5)]);
      Eat();
      release(Fork[Min(i, (i+1) mod 5)]);
      release(Fork[Max(i, (i+1) mod 5)]);
   }
}
```

This solution is a deadlock-free and starvation-free solution.

Dinning Philosophers Problem

Solution 3: Force the system with a global lock (mutex) so that only one philosopher can eat at a given time (has a performance bug).

Solution 4: Uses an arbitrator (waiter), where two forks are requested at a time, i.e., picking both forks in a critical section.

Reduced parallelism: if a philosopher is eating and one of his neighbors is requesting the forks, all other philosophers must wait until this request has been fulfilled even if forks for them are still available.

Solution 5: A mix of right-handed and left-handed philosophers.

Solution 6: Allows philosophers to put down a fork if the other one is not available (may starve due to simultaneous checking).

Dinning Philosophers Problem

Solution 7: William Stallings proposed a solution where at most $n - 1$ philosophers are allowed to sit down around the table at a given time.

This guarantees at least one philosopher may always acquire both forks, allowing the system to make progress.

Solution 8: Randomly wait before picking up the forks:

You may think about obliging philosophers to wait for a random time before trying to grab the forks. This may work fine on certain applications (e.g., IEEE 802.3's CSMA/CD and IEEE 802.11's CSMA/CA) but not on other "critical" applications (e.g., Nuclear power plant system).

Dinning Philosophers Problem

In OSs, this classical problem models a group of concurrent processes (threads) that share multiple resources with mutual exclusive access.


FAX


Process 2


Floppy-Drive


Process 3


DVD-ROM


Process 1


Process 4


QR Scanner


Process 5


Micro


Process 4

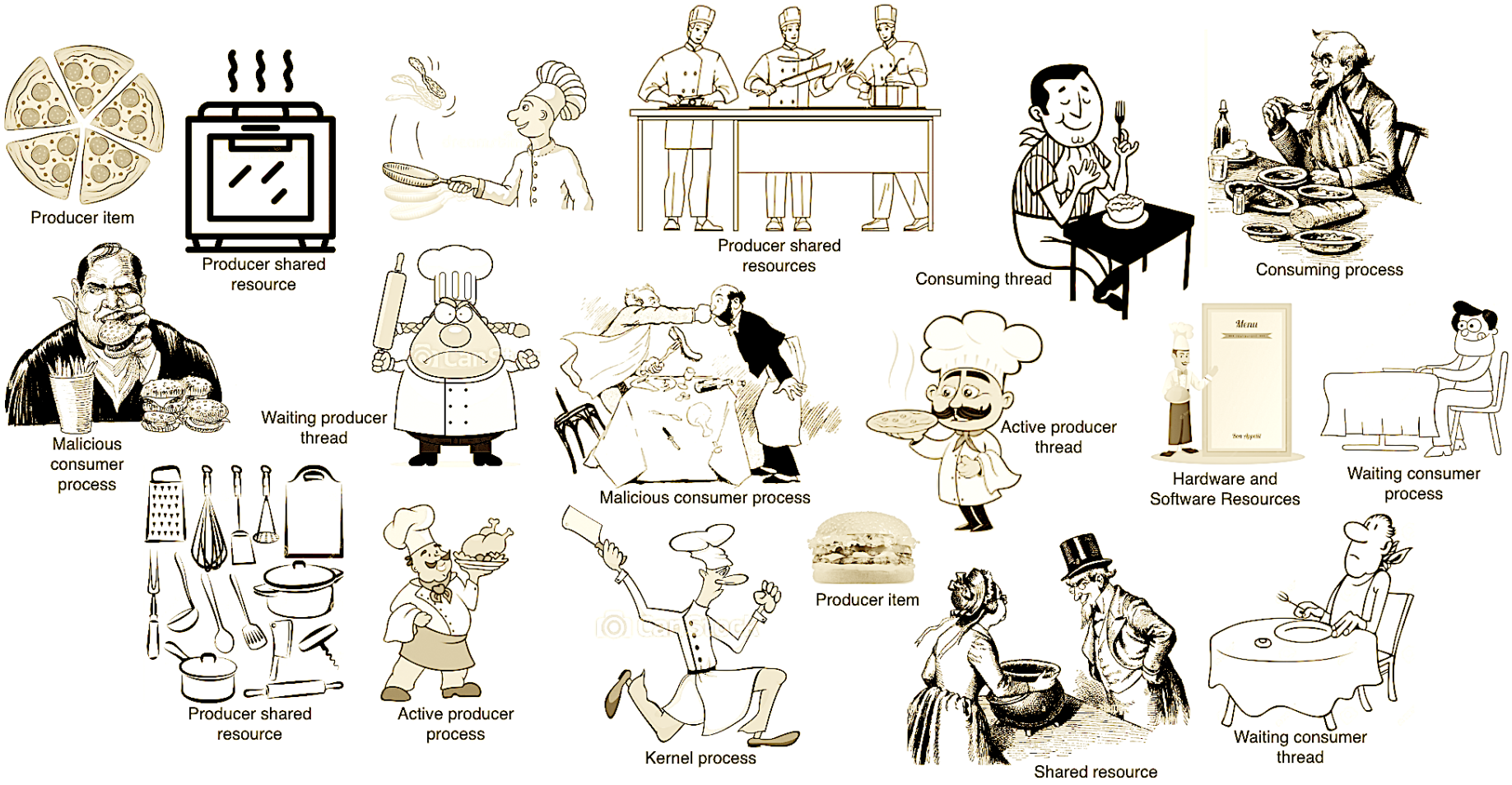

Printer

Dinning Students Problem (Queen's, SC, Winter 2020)

Producers & Consumers Problem

Consumers — Producers Problem

Producers & Consumers Problem



Producers & Consumers Problem

This problem consists of: (1) A set of producer threads $P = \{p_1, \ldots, p_n\}$ & (2) a set of consumer threads $C = \{c_1, \ldots, c_m\}$. Each producer thread p_i produces an item and places it into a buffer B, and each consumer c_i consumes one item from that buffer such that:

- A producer $p_i \in P$ produces an item if the buffer is not full.
- A consumer $c_i \in C$ consumes an item if the buffer is not empty.
- Producers and consumers must have exclusive access on the buffer.
- The buffer B is bounded, i.e., has a limited size $n \in \mathcal{N}$.
- A.k.a., bounded buffer problem.

Producers & Consumers Problem

Solution: One consumer and one producer.

Producer()

```
{
   while(true)
   {

      item = Produce();
      Place(item, B);

   }
}
```

Consumer()

```
{
    while(true)
    {

            Take(B, item);
           Consume(item);

    }
}
```

Producers & Consumers Problem

Solution: One consumer and one producer.

```
Semaphore m = N; Semaphore e = 0; Semaphore b = 1;
```

Producer()

```
{
   while(true)
   {
      acquire(m);
      item = Produce();
      acquire(b);
      Place(item, B);
      release(b);
      release(e);
   }
}
```

Consumer()

```
{
    while(true)
    {
       acquire(e);
       acquire(b);
       Take(B, item);
       release(b);
       Consume(item);
       release(m);
    }
}
```

Producers & Consumers Problem

Solution: Works for multiple consumers and producers, too.

```
Semaphore m = N; Semaphore e = 0; Semaphore b = 1;
```

Producer()

```
{
   while(true)
   {
      acquire(m);
      item = Produce();
      acquire(b);
      Place(item, B);
      release(b);
      release(e);
   }
}
```

Consumer()

```
{
    while(true)
    {
       acquire(e);
       acquire(b);
       Take(B, item);
       release(b);
       Consume(item);
       release(m);
    }
}
```

Viz., this Python script for demonstrtion.

Classical Synchronization Problems

There exists several classical synchronization problems:

- Readers and writers problem.
- Dining Philosophers problem.
- Producers and consumers problem.
- Sleeping barber problem (by Edsger Dijkstra, 1965).
- Cigarette smokers problem (by Suhas Patil, 1971).

We covered three of them, the rest are for you to explore.

✣ You can also think about new synchronization problems that we can put in an exam.

Using Semaphores for Synchronization

The following steps describe an informal procedure of writing codes with semaphores:

1. Understand the problem to solve.

   "Understanding a question is half an answer"-Socrates 470-399 BJC

2. Determine which processes or threads are needed.

   e.g., Readers and writers

3. Write down informally, conditions under which a process must wait for another:

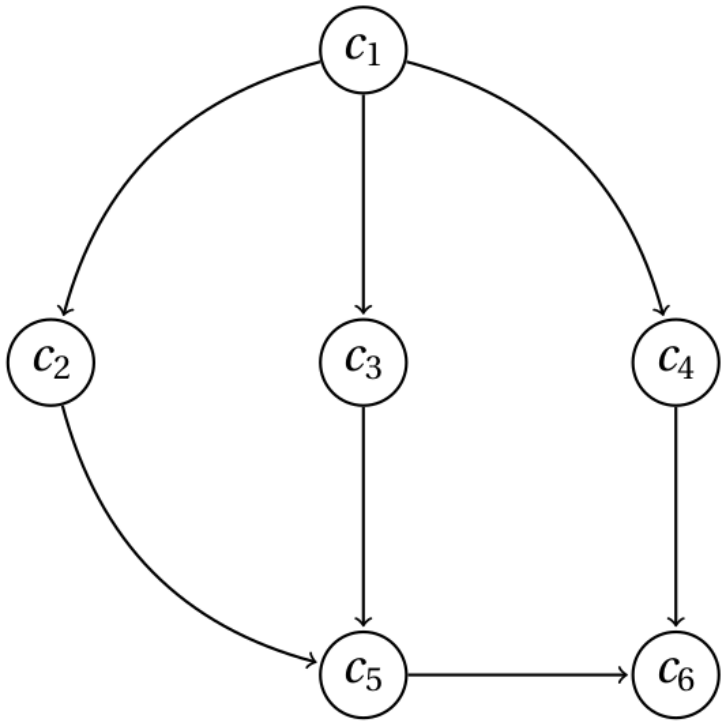
   e.g., A reader waits if a writer is writing

4. Figure out what counters and semaphores are needed.

5. Code and debug.

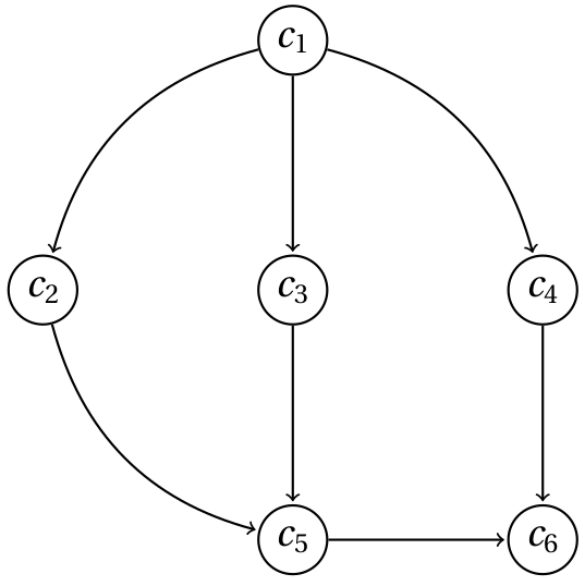
Again: Semaphores better than busy waiting

Exercise. Consider the following PPG. Without using semaphores, write the synchronized code for each process w-r-t the precedence constraints:



Again: Semaphores better than busy waiting

A horrible and inefficient solution could be:



Boolean T1done, T2done, T3done, T4done, T5done = false;

<Code 1> T1done=true;	While (!T1done); <Code 2> T2done=true;	While (!T1done); <Code 3> T3done=true;	While (! T1done); <Code 4> T4done=true;	While (!T2done && !T3done); <Code 5> T5done=true;
While (!T5done && !T4done); <Code 6>				

Again: Semaphores better than busy waiting

Sometimes, you can merge codes together if it is allowed:

Semaphore X, Y = 0;

<Code 1> V(X); V(X); V(X); P (X); <Code 2> V(Y) P(Y) P(Y) <Code 5> V(X) P(X) P(X) <Code 6>	P(X); <Code 3> V(Y);	P(X); <Code 4> V(X);

Semaphore X, Y = 0;

<Code 1> V(X); V(X); <Code 2> P(Y) <Code 5> P(X) <Code 6>	P(X); <Code 3> V(Y);	P(X); <Code 4> V(X);

Again: Semaphores better than busy waiting

Sometimes, you can merge codes together if it is allowed:

Semaphore X, Y = 0;

<Code 1> V(X); V(X); V(X); P (X); <Code 2> V(Y) P(Y) P(Y) <Code 5> V(X) P(X) P(X) <Code 6>	P(X); <Code 3> V(Y);	P(X); <Code 4> V(X);

Semaphore X, Y = 0;

<Code 1> V(X); V(X); <Code 2> P(Y) <Code 5> P(X) <Code 6>	P(X); <Code 3> V(Y);	P(X); <Code 4> V(X);

Watch out! this code is incorrect. A precedence constraint is broken.

- End.